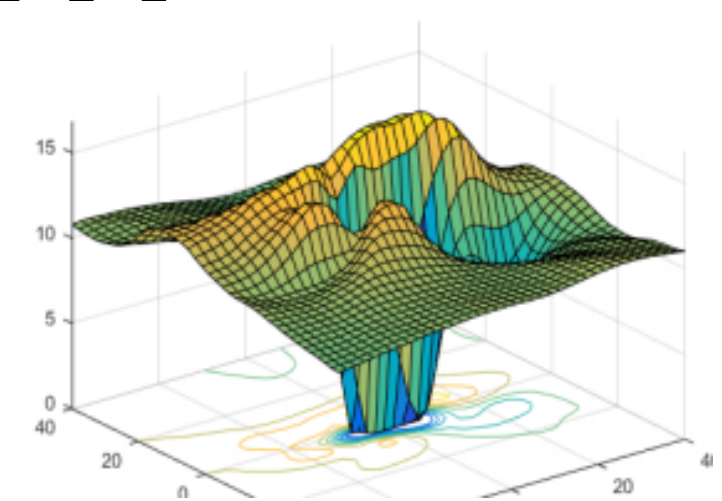


Adversarial Robustness: Geometrical Perspective

Ruoyu Sun
ISE Department, Coordinated Science Lab
UIUC

Oct 10, 2019. Guest lecture at AML class by Bo Li

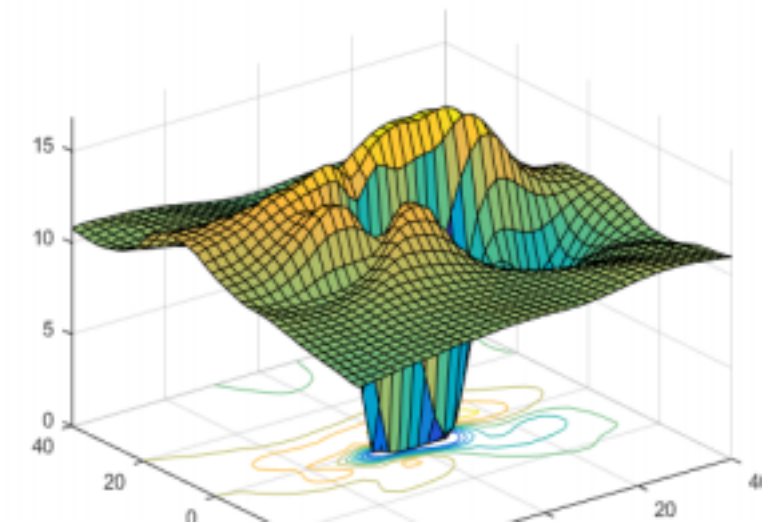
Motivation



- Why **geometry**?
- Geometry: one of the main pillars of math (geometry, algebra, analysis)
- Better “**understanding**” of the problem
- Adversarial robustness: a mysterious subject in ML.
—maybe geometry helps?

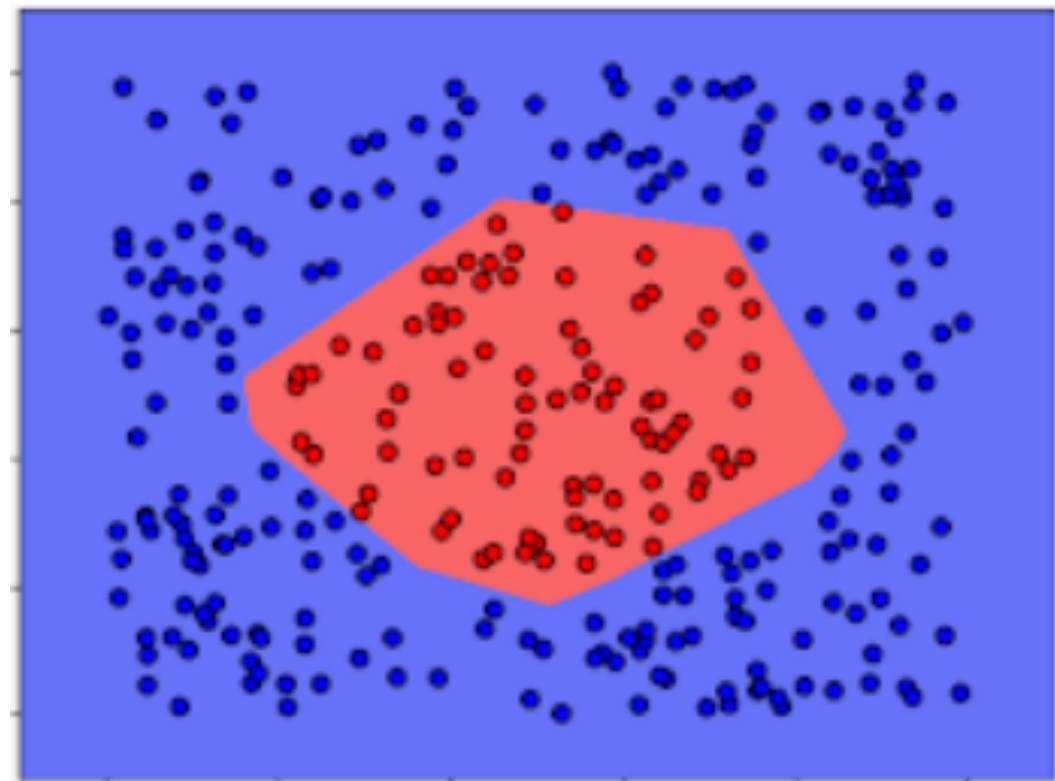
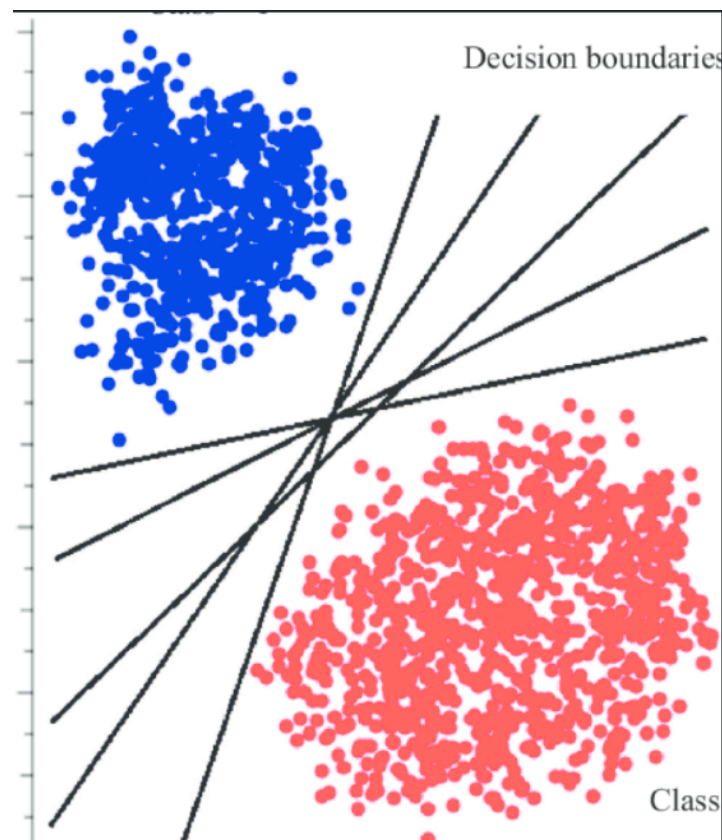
This lecture

- Looking at robustness from geometrical perspective
- 1) Explaining robustness by landscape
- 2) Random smoothing, and geometry
- 3) Mode connectivity and robustness



Part I Basic Geometry of Robustness

Simple Visualization of Classification

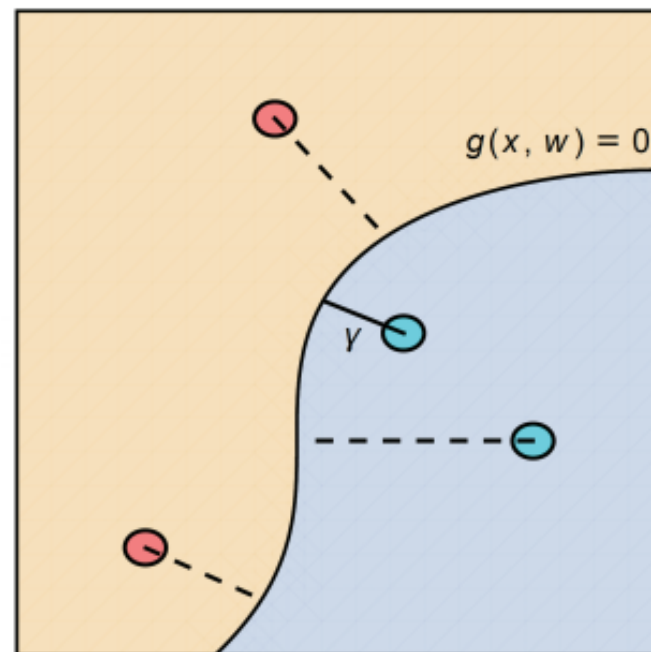


Two (or more) classes of data

A classifier, i.e., decision boundary (**DB**)

DB Example 1: Robust Training

- Small perturbation causes wrong prediction means what?
Answer 1: crossing the decision boundary



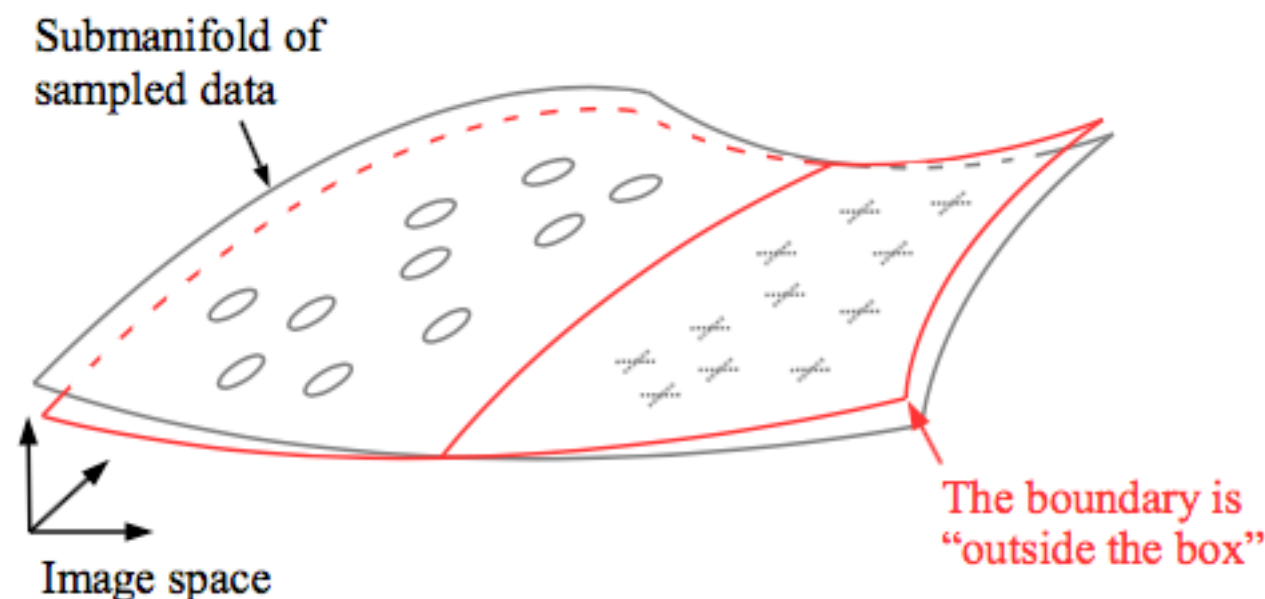
- “Folklore” solution: [large margin training](#) (only work for perturbation)
- e.g. Large Margin Deep Networks for Classification, NeurIPS’18, Google.

DB Example 2: Explaining Adversarial Paradox

Fundamental question: Why does CNN has test error $< 5\%$ on ImageNet, but has robust error 100%?

Tanay, Griffin (UCL), 2016, A Boundary Tilting Perspective on the Phenomenon of Adversarial Examples

Hypothesis: DB extends beyond the sub-manifold of sample data and lie close to it.



DB Example 3: Universal Perturbation

Moosavi-Dezfooli et al., 2017: Analysis of universal adversarial perturbations

- Provide an analysis on the existence of “universal perturbations vector” that fools all instances
- Linear DB: possible
- Main intuition: positive curvature ==>vulnerable

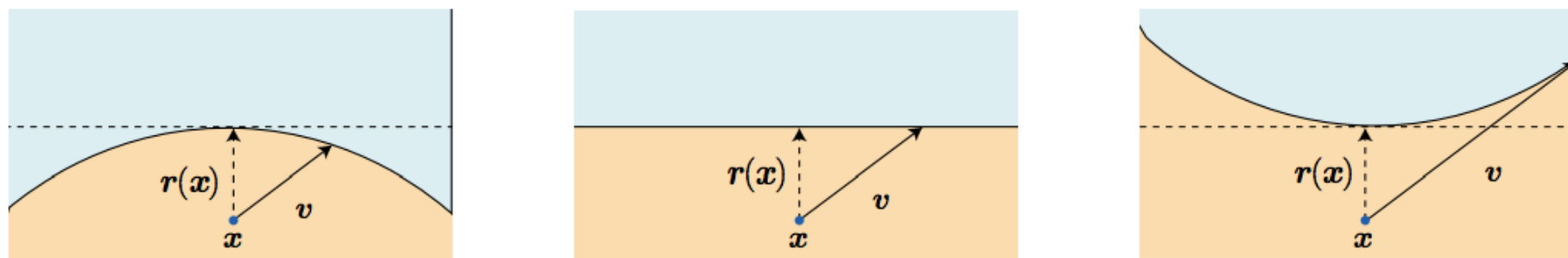
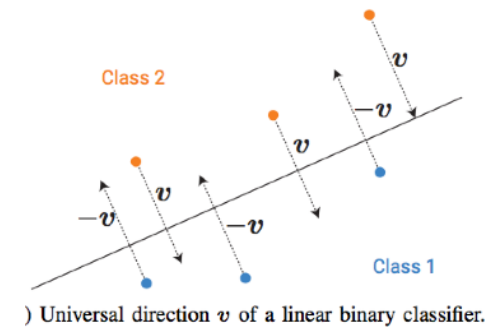


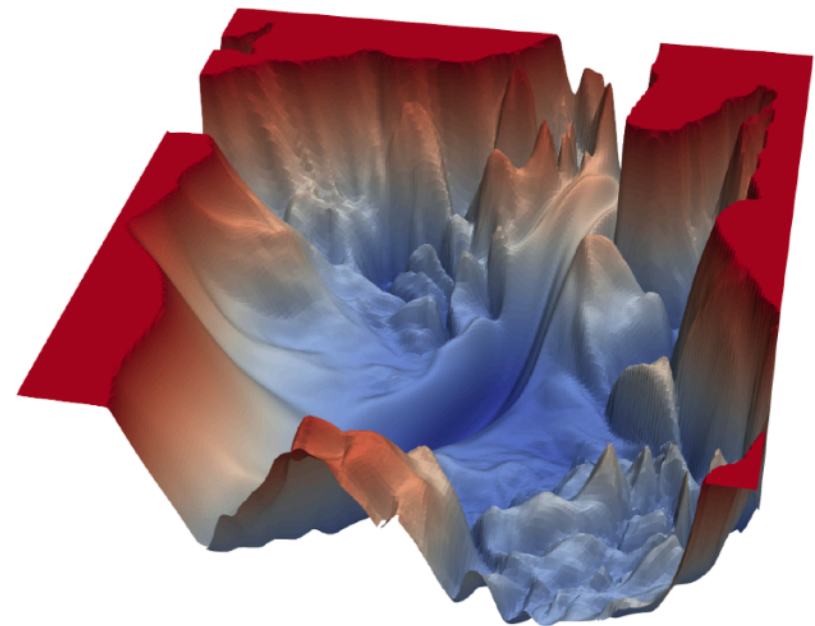
Fig. 3: Link between robustness and curvature of the decision boundary. When the decision boundary is *positively* curved (left), small universal perturbations are more likely to fool the classifier.

Beyond DB: Loss Landscape

Many of the understandings come from DB; what else in geometry can we say?

Loss landscape is popular for understanding neural-net training.
 $\min_w F(w)$.

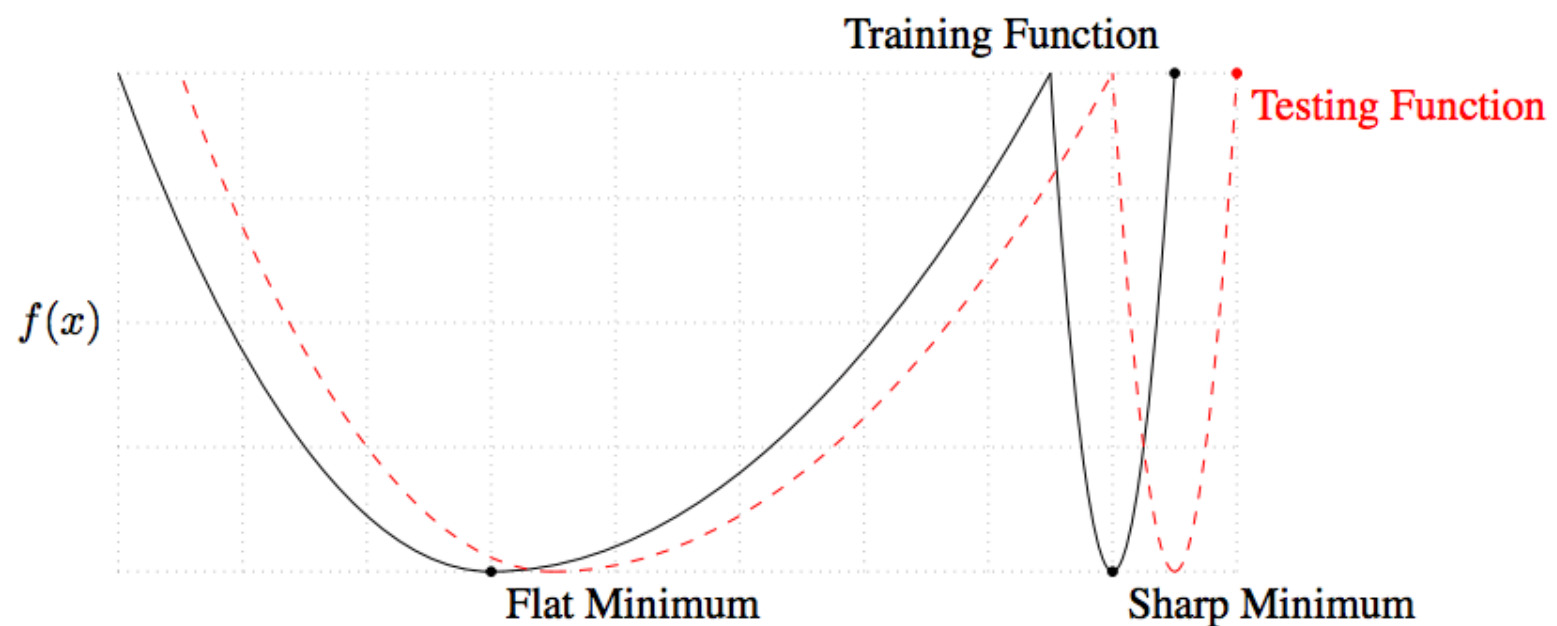
Loss landscape is the image $(w, F(w))$;
Often visualized for w in $\mathbb{R}^2$.



“Wide” Minima Generalize Better?

Keskar et al., 2017: On large batch training for deep learning: Generalization gap and sharp minima.

Intuition?

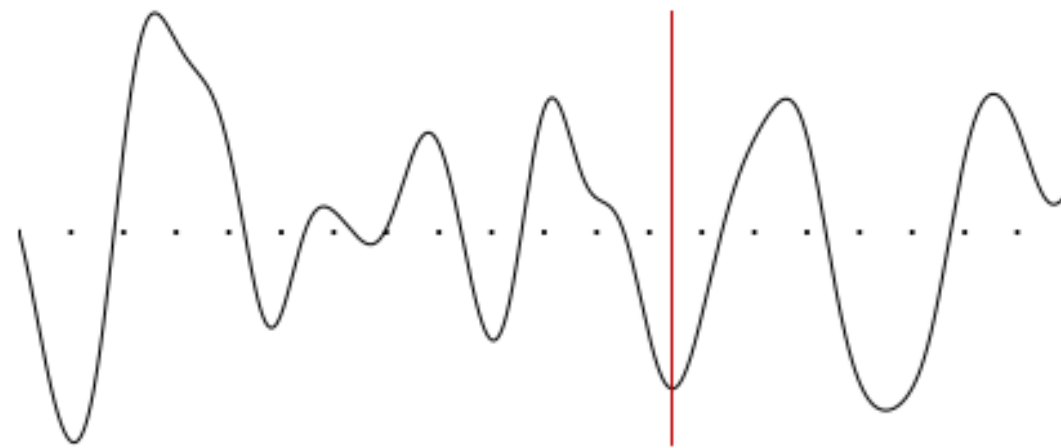


“Wide” Minima Generalize Better?

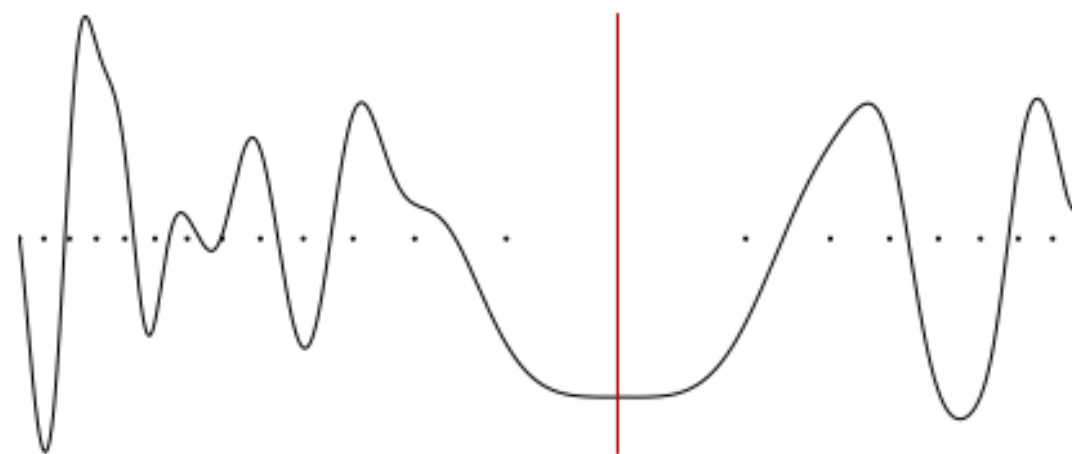
Dinh et al., 2017: Sharp Minima Can Generalize For Deep Nets

Intuition:

re-parameterization
exists, making notion
of “wide” ill-defined



(a) Loss function with default parametrization



(b) Loss function with reparametrization

“Wide Minima” More Robust?

To connect “wide” minima with robustness, use a simple example.

My claim: “Wide” minima roughly correspond to large margin.

(use the board)

Lesson: correlation of data space and parameter space. (use board if possible)

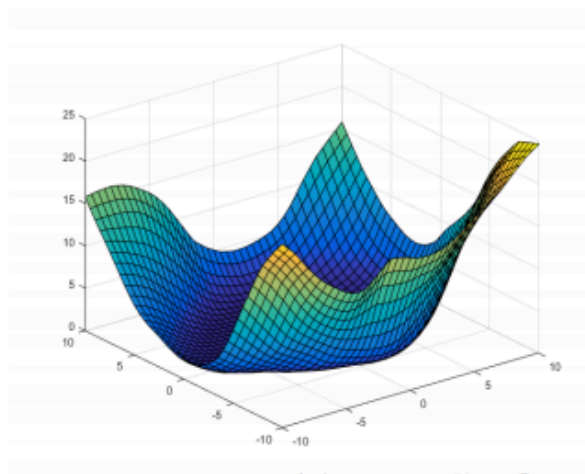
“Wide Minima” More Robust?

First, possible, “wide” is related to large-margin DB.

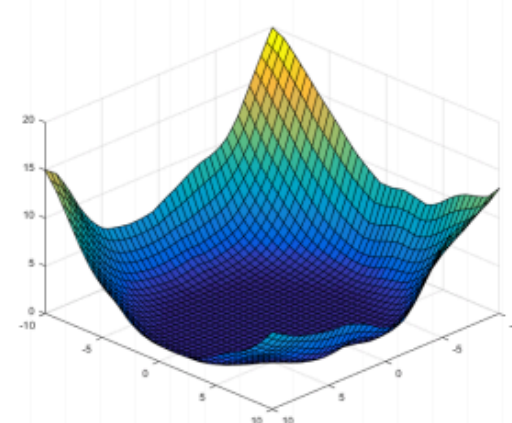
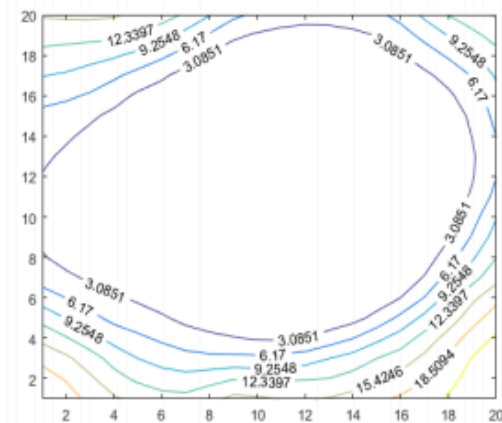
Second, problematic. Seems do not distinguish generalization and robustness.

Third, negative numerical evidence exists (not strong).

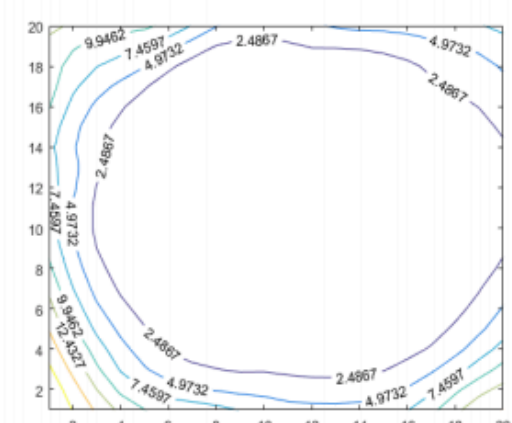
Yu et al., 2019, INTERPRETING ADVERSARIAL ROBUSTNESS: A VIEW FROM DECISION SURFACE IN INPUT SPACE



(a) Loss Surface of Natural Model



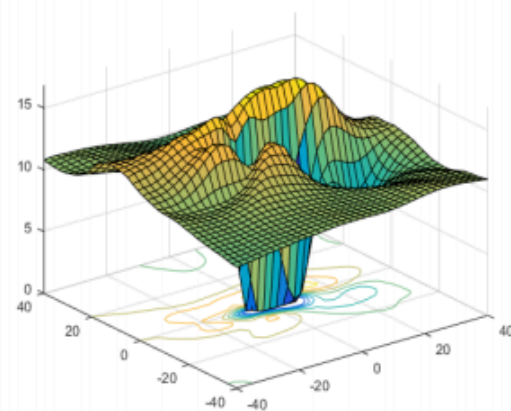
(b) Loss Surface of Robust Model



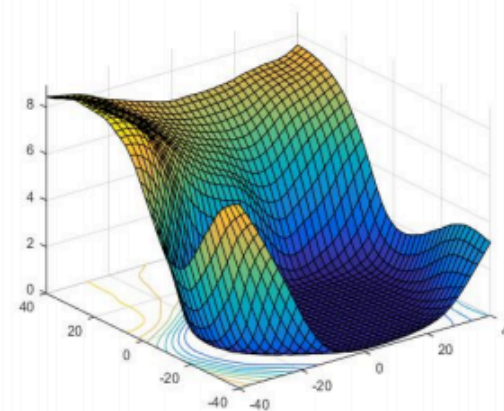
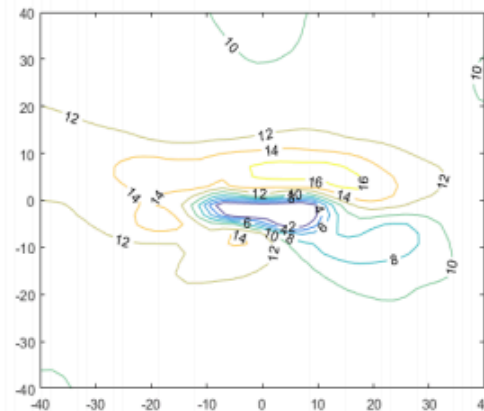
Coming Back to Data Space

Yu et al., 2019, INTERPRETING ADVERSARIAL ROBUSTNESS: A VIEW FROM DECISION SURFACE IN INPUT SPACE

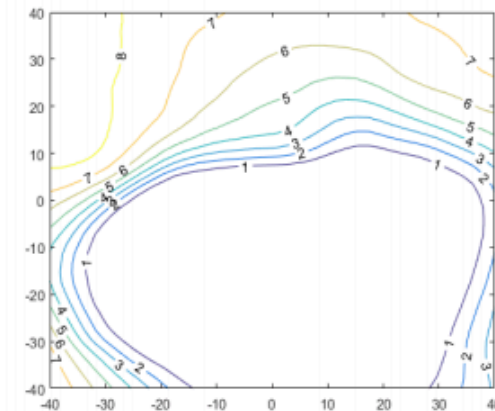
Shows the visualization in data space.



(a) Loss Surface of Natural Model



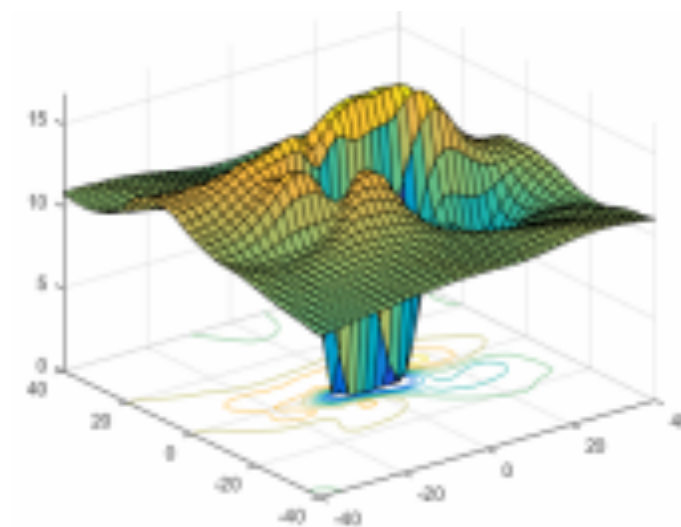
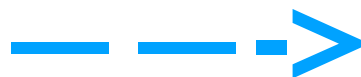
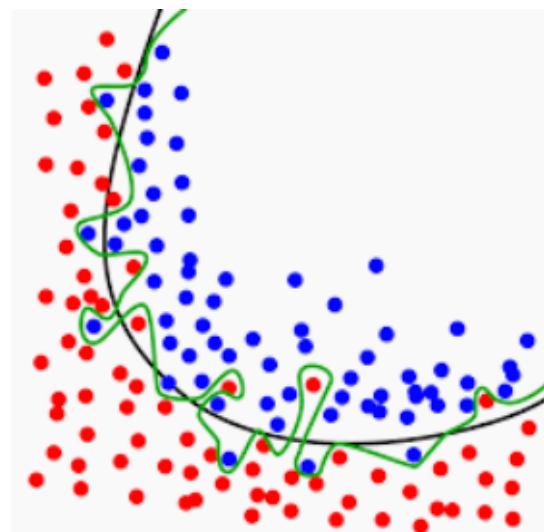
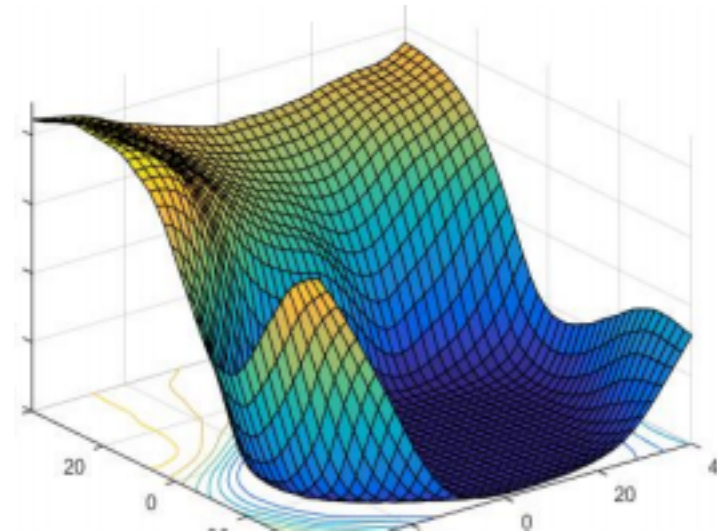
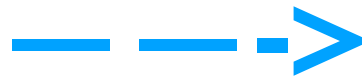
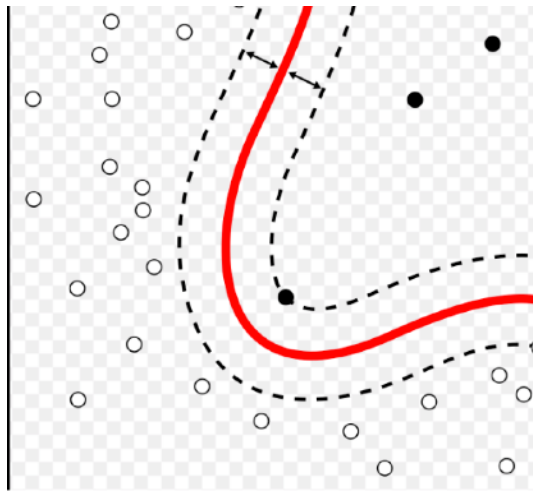
(b) Loss Surface of Robust Model



Loss Surface in data space. **Smother ==> More robust.**
Somewhat trivial claim...

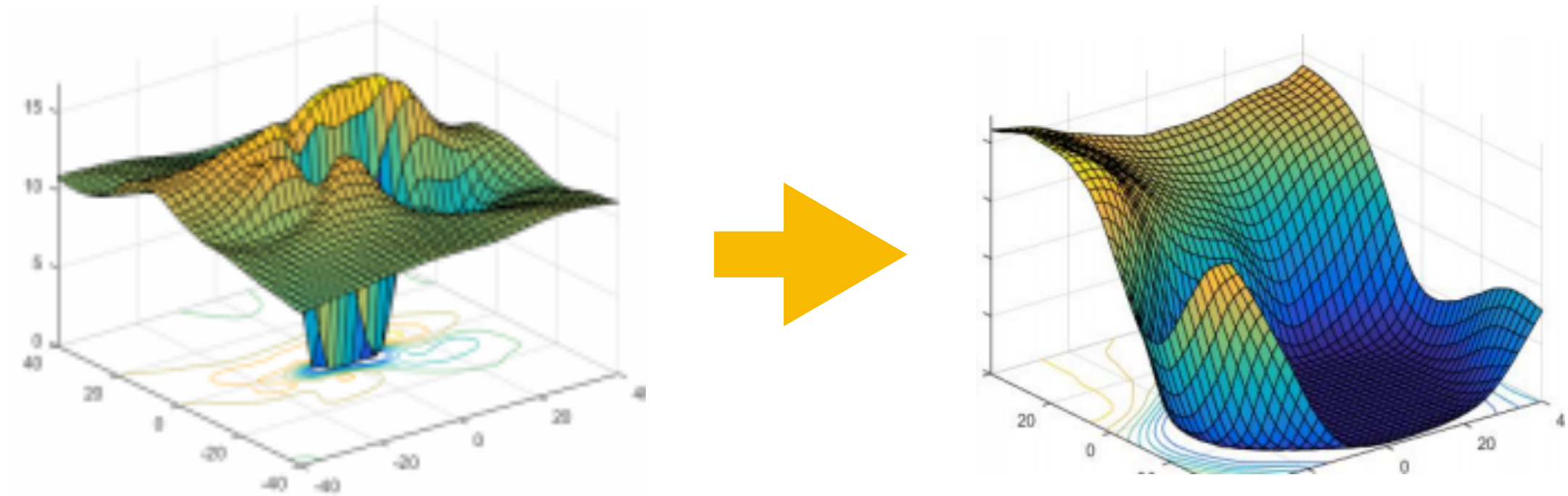
Data Space Smoothness

Data space + DB $\longrightarrow$ (Data space, prediction value) image



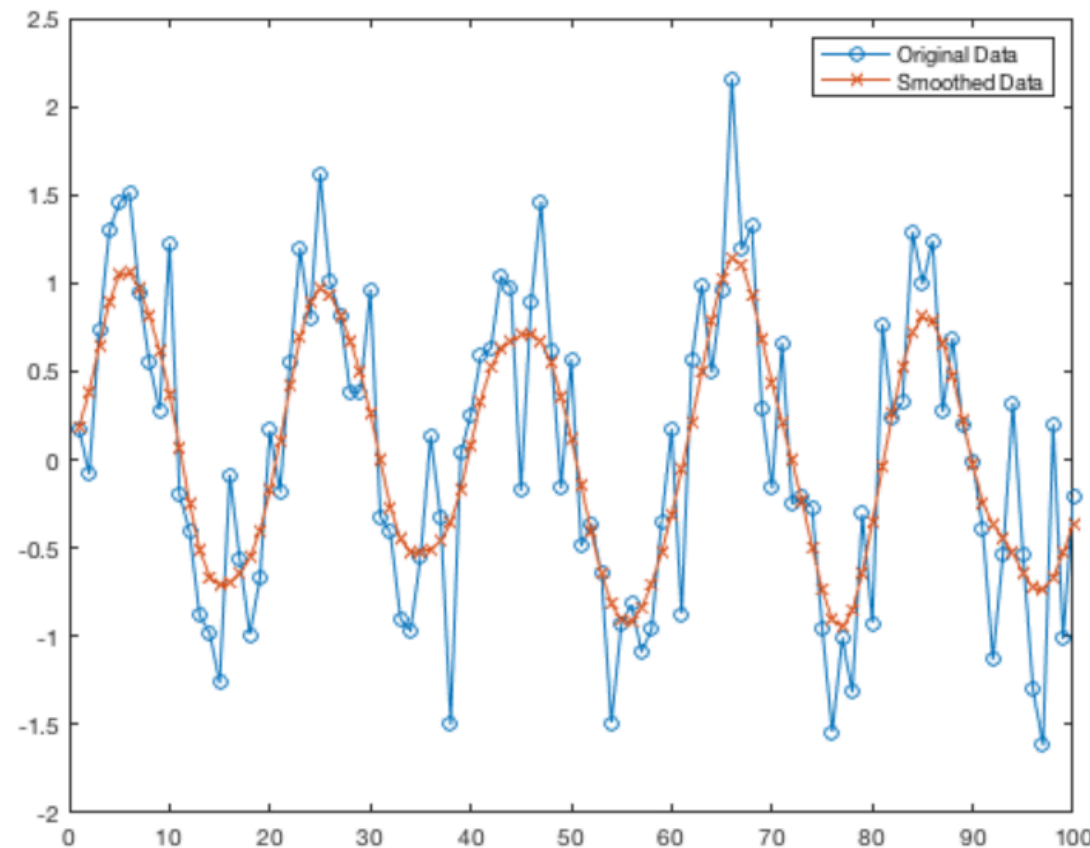
Part II Random Smoothing

Smoothing



How to make the landscape of a function smoother?

Smoothing: Classical Math Problem



- Various ways to smooth a function: e.g. moving average, kernel smoothers, noise (e.g. Kleinberg et al. 2018: SGD is working on a convolved (thus smoothed) version of the loss function)

Math of Smoothing: 1d

- **Local-averaging** smoother: given a function $f(x)$, define

$$g(x) = \frac{1}{2\delta} \int_{x-\delta}^{x+\delta} f(u) du .$$

Here $g(x)$ is the “average” value of f in a neighborhood of x .

- **Kernel** smoother: a **window**, and a kernel (**filter**)

$$g(x) = \frac{1}{B(x)} \int_{x_{min}}^{x_{max}} K(x - u) f(u) du .$$

- e.g. $K(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2} \frac{x^2}{\sigma^2}\right) .$

- **General smoother**: a **window**, and a function **h** on all values of the window

Robust Classification

For simplicity, consider binary classification.

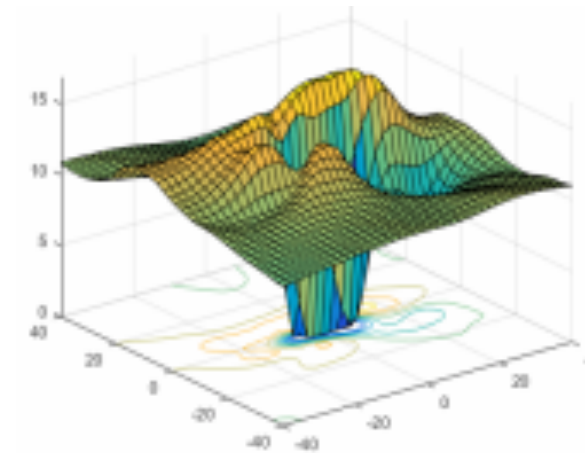
Given data $x_1, x_2, \dots, x_n \in \mathbb{R}^d; y_1, \dots, y_n \in \{1, -1\}$, trained a classifier f : $f(x)$ is 1 or -1.

e.g. $f(x) = \text{sign}(F(x))$, where F is neural-net.

Very likely, f is not robust. **How to make it robust?**

Natural idea: **smoothing**

Which Smoothing?



Averaging? Possible choice.

Note f is discrete function, output 1 or -1. Averaging, then round to 1 or -1.

Equivalent to: “**voting**”.

Voting: in neighborhood, **if more 1 than -1, then output 1**; otherwise, output -1.

Smoothing by Voting

Voting-smoother: in neighborhood, if more 1 than -1, then output 1.

Consider a ball around x with radius σ , $B(x, \sigma)$

Let $\mathbf{Unif}(B(0, \sigma))$ be the uniform distribution in the ball.

$$g(x) = \begin{cases} 1, & \mathbb{P}(f(x + \epsilon) = 1) > 1/2 \\ -1 & \text{otherwise.} \end{cases}$$

More general, can be any distribution around x ! (e.g. Gaussian)

Certified robustness for smoothed classifier

Certified robustness: for a classifier g , compute a quantity $q(x)$: if $q(x) > 0$, then $g(x)$ is robust at x , i.e.,

$$g(x) = g(x + \epsilon), \forall \epsilon \in B(0, \sigma). \quad (*)$$

Important requirement: computing $q(x)$ is feasible.

Naive method; verifying $(*)$ directly. **Impossible** in finite time.

LP-duality based: open the blackbox of g , utilizing other properties.

Rigorous Result

Cohen et al. 2019, Certified Adversarial Robustness via Randomized Smoothing.

Use Gaussian distribution, instead of uniform (for computation ease).

Let $g(x) = \arg \max_{c \in \{1, -1\}} P(f(x + \epsilon) = c)$, **where** $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$.

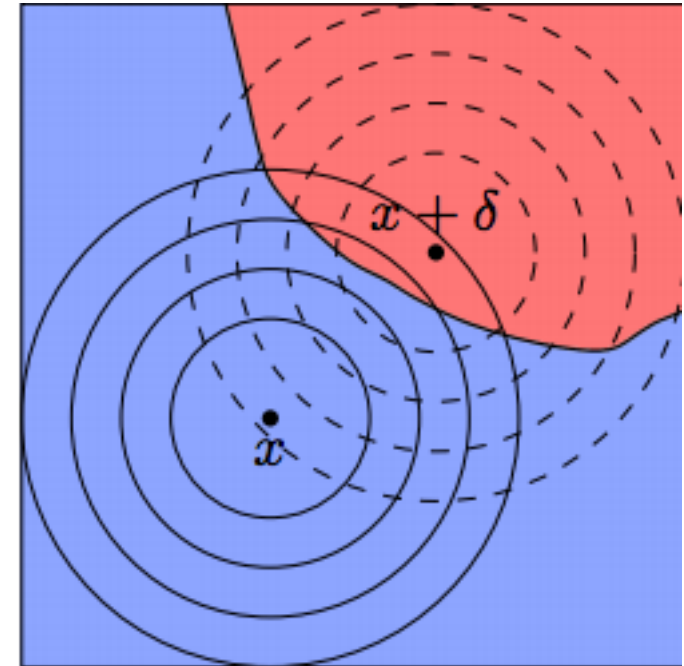
Theorem 1: Suppose $P(f(x + \epsilon) = c) \geq p_A > 1/2$, then

$$g(x') = c, \quad \forall \|x' - x\| < \sigma \Phi^{-1}(p_A).$$

Here $\Phi(t)$ is the CDF of standard Gaussian distribution.

Certified robustness for smoothed classifier

Nice property of smoothed classifier:
possible to certify robustness!



Intuition: $g(x)$ and $g(x')$ are not independent — they depend on voting. Their **voters have large overlap!**

Claim: If $\mathbb{P}(f(x + \epsilon) = 1) > 1/2 + p$, (1) (majority is 1)
 $\mathbb{P}(x + \epsilon \notin B(x', \sigma)) < p$, (2) (at most p -portion changes)
Then $g(x) = g(x')$.

Proof: One line.

Large Overlap

$$\mathbb{P}(x + \epsilon \in B(x', \sigma)) < p, \quad \forall \|x' - x\| < \delta.$$

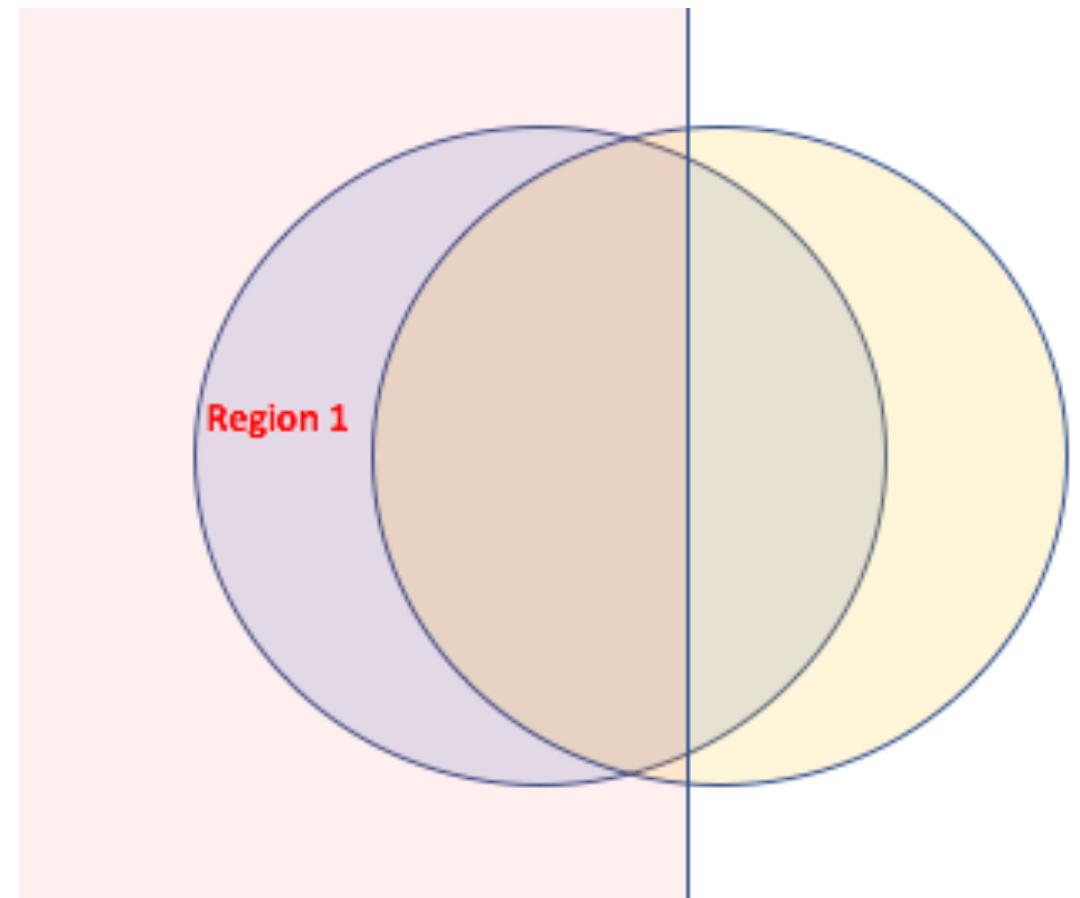
i.e. $\mathbf{Vol}[B(x, \sigma) \cap B(x', \sigma)] / \mathbf{Vol}[B(x, \sigma)] < p$

Main intuition: **region 1 is small!**

Two unit disks.

Moving distance δ . $\theta = \arccos(\delta/2)$

$$\text{Area (region 1)} = \pi - 2\theta + \sin(2\theta)$$



Precise number not that important, as long as it is **computable**.

Certified robustness for smoothed classifier

To ensure robustness, need:

$$(1) \quad \mathbb{P}(f(x + \epsilon) = 1) > 1/2 + p,$$

Verifiable by simulation!

$$(2) \quad \mathbb{P}(x + \epsilon \in B(x', \sigma)) < p, \quad \forall \|x' - x\| < \delta.$$

Not verifiable, since “for all” x' .

But **provable**! (for certain choices of parameters)

Intuition: moving a ball slightly, will still have large overlap.

Final Step: Estimating Probability

How to ensure $\mathbb{P}(f(x + \epsilon) = c) > p_A$?

Question: given a Bernolli coin with $P(\text{head}) = p$, how to estimate p ?

Remark: not about high-dim! Just **1-dimensional Bernolli!**

Method: Throw it for enough times, then get a good estimate of p , with **high probability!**

Can easily get a lower bound on p , w.h.p.

Probabilistic Verification

Important concept: **probabilistic verification**.

Other certified robustness is “deterministic verification”: if you get $q(x) > 0$, then f is **surely** robust around x .

Here, if you get $q(x) > 0$, then f is robust around x with **prob** at least, say, $1 - 0.00001$.

Similar to any randomized algorithms, **prob is controlled by you**, not by environment! (different from buying lottery ticket)

Acceptable?

Probabilistic verification is rather acceptable.

...not so acceptable, if you are rigorous mathematicians, or TCS researchers on pseudo-randomness.

Extension

Extension to multi-class is similar.

Is Gaussian smoothing critical?

Is l_2 distance critical?

What is the trade-off between σ (checked ball size around f), and the radius in the definition of $g(x')$? (currently, the same, but not necessary)

Very General Framework

In summary: **very general framework**. (lots of things optimizable)

Challenges for generalization:

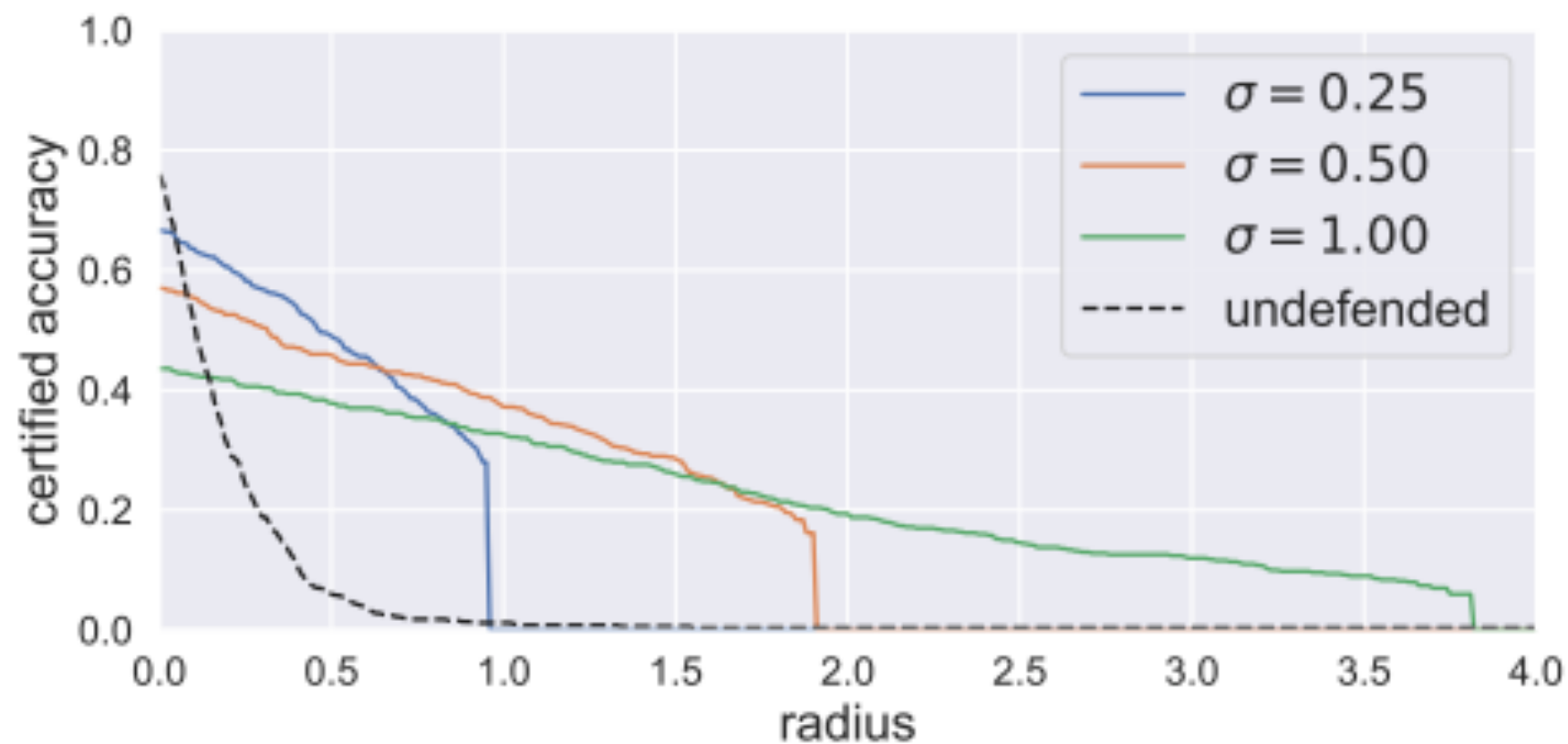
- Proof of step (2) is non-trivial;
- Simulation of step (1) unknown.

Good Simulation Results

ImageNet, ResNet 50. (almost the only one **scalable to ImageNet**)

Certifying each image on RTX 2080Ti, takes 110 sec.

Trade-off between accuracy and certified radius.



Part III Exploring Parameter Space Landscape

Exploring Parameter Space

We have discussed the data space geometry.

Next, we show that exploring parameter space is also useful.

I had this idea a few months ago, but noticed an ICLR2020 submission already did this:

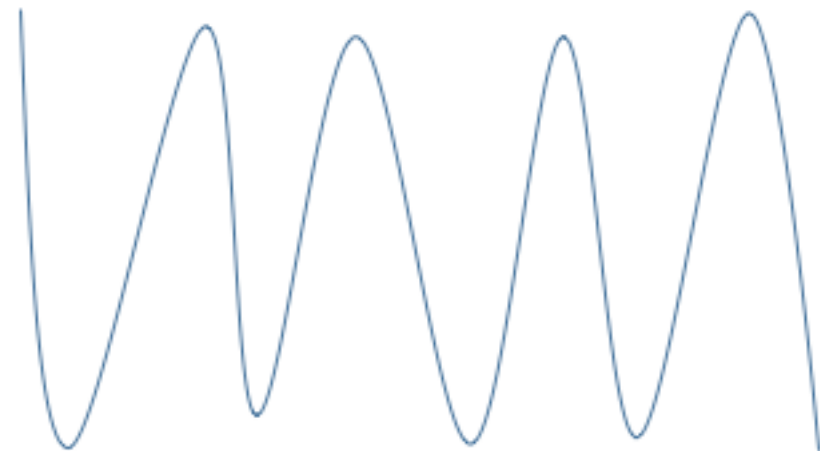
BRIDGING MODE CONNECTIVITY IN LOSS LANDSCAPES
AND ADVERSARIAL ROBUSTNESS,

Background: mode connectivity

What is a typical landscape of neural-net loss?

Advertisement: **Optimization theory for deep learning: an overview**,
Ruoyu Sun, forthcoming.

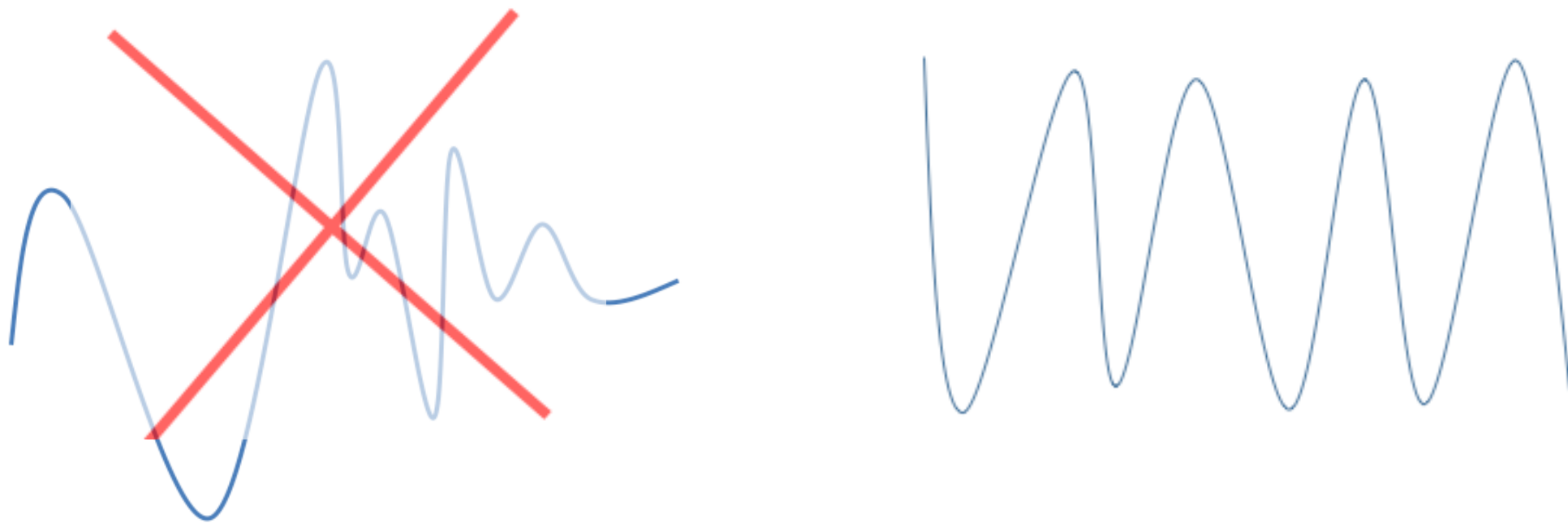
Many bad local-min, due to non-convexity? or no bad local-min?



Background: mode connectivity

What is a typical landscape of neural-net loss?

Many bad local-min, due to non-convexity? or no bad local-min?



Li, Ding, Sun'2018: over-parameterized nets have **no bad basin**.

Partially explains why different initial points can converge to similar loss values.

Background: mode connectivity

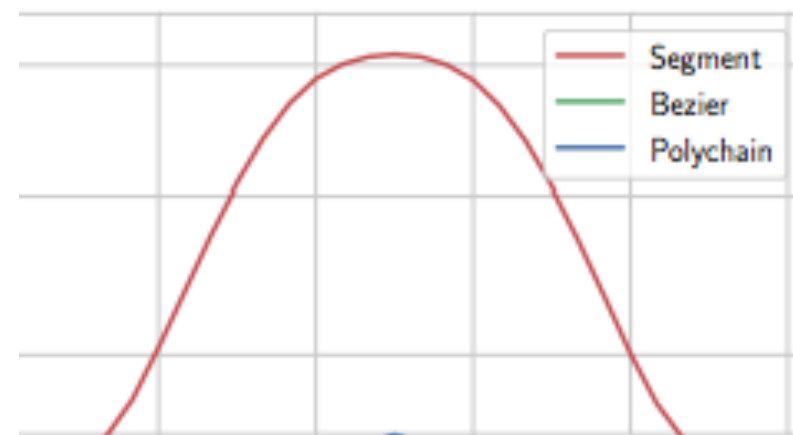
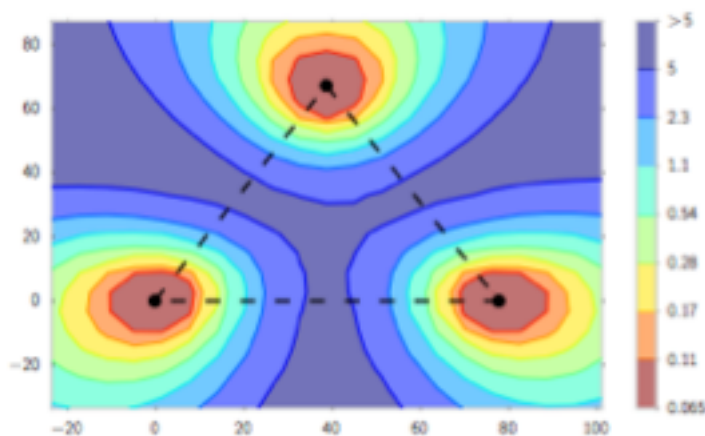
Garipov et al. Loss Surfaces, Mode Connectivity, and Fast Ensembling of DNNs, NeurIPS'2018. Also, see, Draxler et al. 2018.

Train a neural-net, from three random initial points, get three points

$$w_1^*, w_2^*, w_3^*$$

Left figure: connecting two of them, get $w(t) = tw_1^* + (1 - t)w_2^*$

Evaluate $f(w(t))$, $t \in [0,1]$, will see a bump (high training error).



Background: mode connectivity

Bezier curve: for a mid-point θ , the path

$$w_{\theta}(t) = (1 - t)^2 w_1^* + 2t(1 - t)\theta + t^2 w_2^*$$

Not every θ gives a path that is nice.

Solve a new problem to find θ ; i.e., find a path in this family of paths:

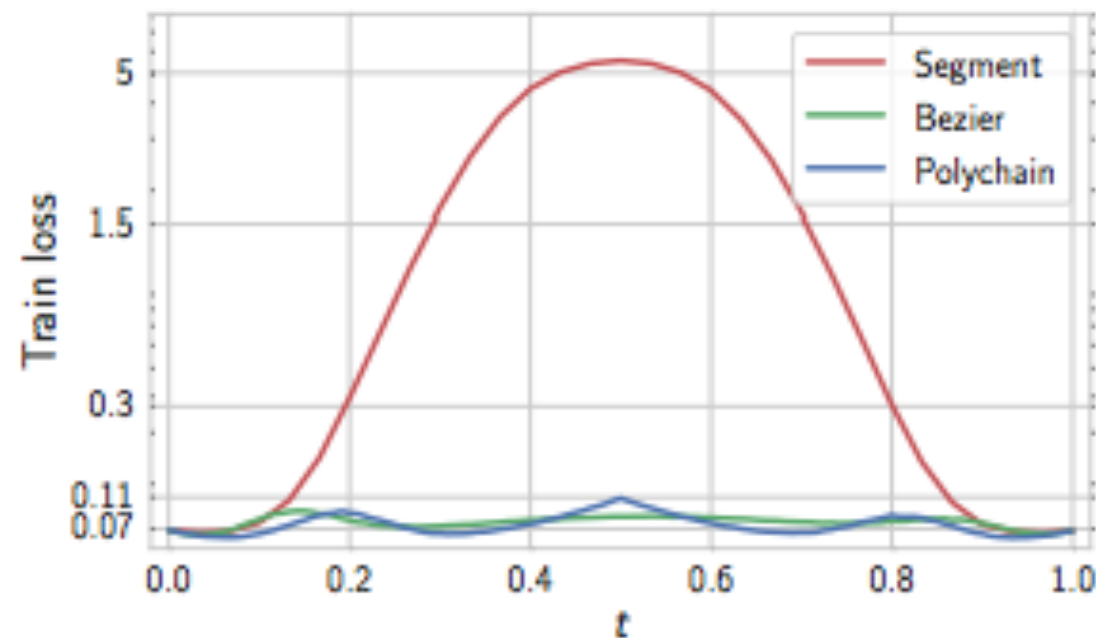
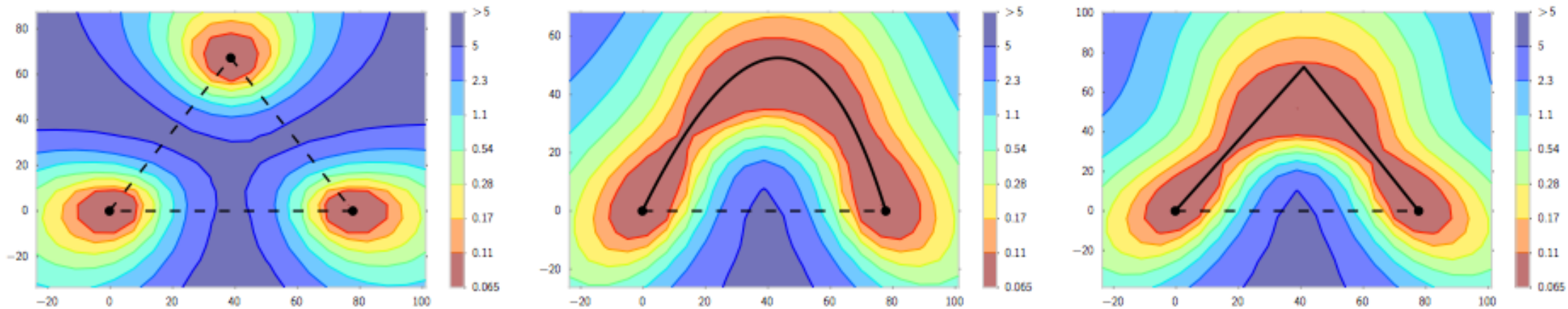
Goal: want the values of f along the paths are as small as possible. (all are small)

$$\min_{\theta} \frac{1}{n} \sum_{i=1}^n \int_{t=0}^1 f(w(t); x_i) q_w(t) dt .$$

$$q_{\theta}(t) = \|w'_{\theta}(t)\| \left(\int_0^1 \|w'(\theta)(t)\| dt \right)^{-1} .$$

Background: mode connectivity

Then evaluate $f(w(t)), t \in [0,1]$. Can be all close to minimum!



Background: mode connectivity

Empirical Claim: for any two “global minima”, there is a path connecting them with error close to the minimum (e.g.. 5% error).

“Global-min”: converged points from two random initial points.

Why is it useful?

Give us a way to **find many global minima!** (with small test error)

Compare to naive way: to find 1000 global minima, need to run algorithm 1000 times.

Mode connectivity for robustness?

Lesson: a way to **find many global minima!** (with small test error)

Again, why is this lesson useful?

Well, for benign training, not so much — already find a good solution, why bother to find 1000 more?

For robustness: need to both **small test error**, and **small robust error**

Method (conceptual):

- (1) Restricted to the class of small-test-error solutions, S .
- (2) Find **one solution in S** that is the **most robust**.

Defending backdoor attacks

Follow the procedures in (Gu et al., 2019).

Implement backdoor attacks; obtain two backdoored models trained on the same poisoned training data.

10% of the training data are poisoned by inserting the trigger and changing the original correct labels to the target label(s)

	Backdoor attacks Dataset	Single-target attack	
		CIFAR-10 (VGG)	SVHN (ResNet)
Model ($t = 0$)	Clean images	15%	5.4%
	Triggered images	0.07%	0.22%
Model ($t = 1$)	Clean images	13%	7.7%
	Triggered images	2%	0.17%

Finding Path: What Data

How to find the path? Slightly different from mode-connectivity paper.

Garipov18 uses all training data to find the path.

ICLR20 paper:

Step 0: Assume the model is already pre-trained (perhaps with back-door data).
Want to repair, and use on test data set.

Step 1: **use a set of bonafide data (subset of test data) to find the path.**
(Remark: somewhat surprising! **Much fewer data than all training data**)

Step 2: test the accuracy (along the path) using other test data (5000 in experiments).

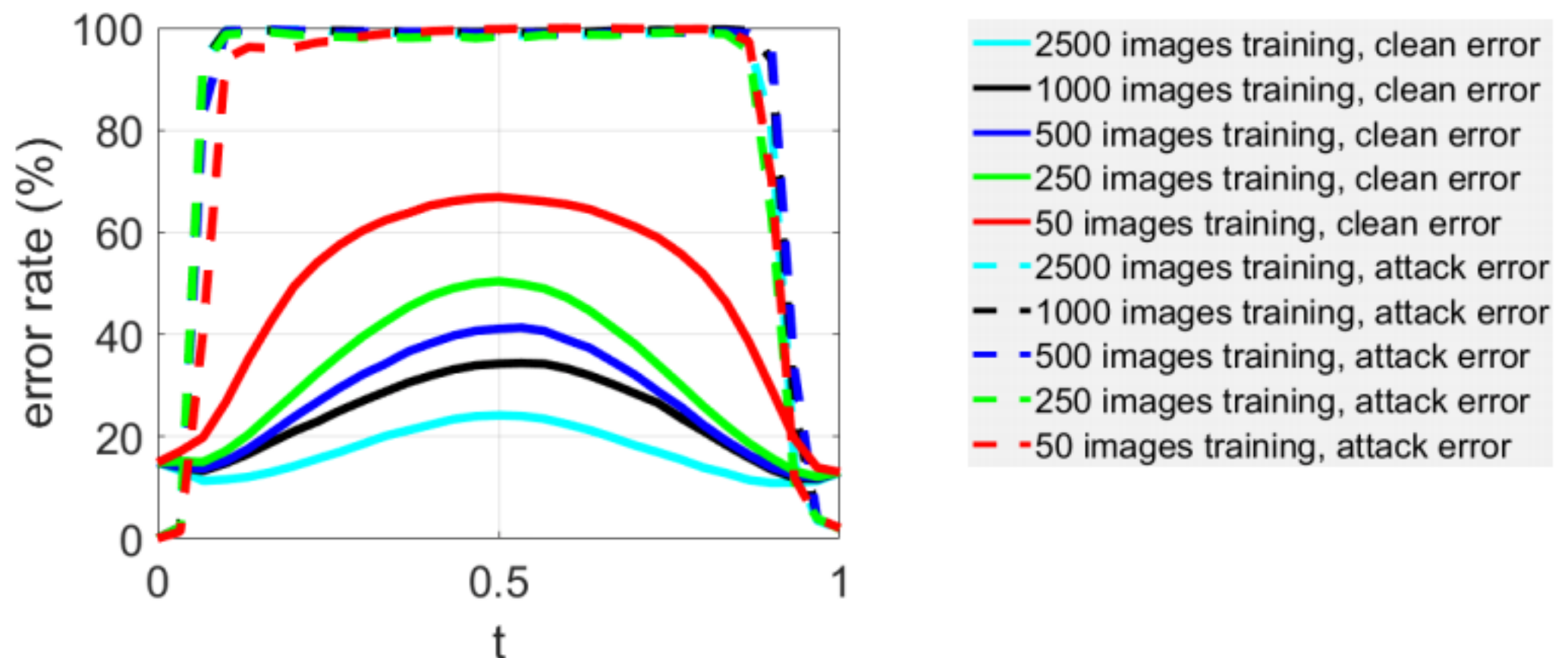
Step 3: use one of the model parameter in the path as the final model (e.g. for production).

Empirical Experiments

As a first step, not much needs to be done: **just check the paths!**

Very effective in repairing backdoor attack (explanations later)

Single-target backdoor attack



Observations From Figure

From the above figure: most of the path is robust!

- (1) Models at $t = 0.1$ or $t = 0.9$ can simultaneously:
get good test acc +
reduce backdoor attack rate from 99% to nearly 0%.
- (2) Most models on the path (e.g. when $t \in [0.1, 0.9]$) exhibit high resilience to backdoor attacks
- (3) Test acc related to amount of bonafide data.

Compare to Baseline

		Method / Bonafide data size	2500	1000	500	250	50
SVHN (ResNet)	Clean Accuracy	Fine-tune	96%	94%	91%	89%	76%
		Train from scratch	87%	75%	61%	34%	12%
		Noisy model ($t = 0$)	13%	13%	13%	13%	13%
		Noisy model ($t = 1$)	11%	11%	11%	11%	11%
		Prune	96%	95%	93%	91%	89%
	Backdoor Accuracy	Path connection ($t = 0.2$)	2.5%	3%	3.6%	4.3%	16%
		Fine-tune	14%	7%	29%	63%	60%
		Train from scratch	3%	3.6%	5%	2.2%	3.9%
		Noisy model ($t = 0$)	51%	51%	51%	51%	51%
		Noisy model ($t = 1$)	42%	42%	42%	42%	42%
		Prune	80%	90%	88%	92%	94%

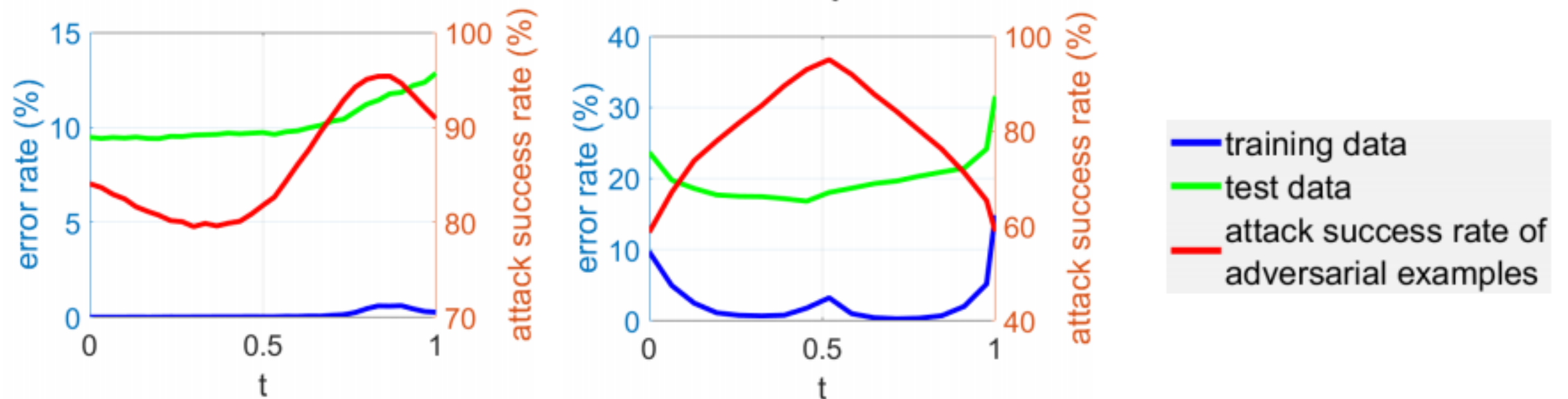
Better.

Better clean acc, or better backdoor acc, or both

(**10%** improvement for 2500 data, more than **50%** improvement on 50 data)

Evasion Attack Experiments

In short, not much improvement, in terms of robust acc.



Connecting regular models Connecting adv-trained models

Ignore details here.

- Reason 1: adv examples can transfer across models.
- Reason 2: have not explored the landscape well.

Concluding Remarks

Geometrical Perspective

- To understand the strange behavior of adversarial attacks, geometry can provide much insight.
- Three tools:
 - decision boundary (and margin)
 - data space landscape
 - parameter space landscape

Two Methods

- Data space: **smoothing**.
 - Certified defense (in probabilistic sense).
 - One of the rare method to scale to ImageNet.
- Parameter space: **explore mode connectivity**
 - Effective for defending backdoor.

Combining Two Views: Min-max Landscape

- Yet another view: min-max problem

$$\min_w \sum_i \max_{x'_i \in B(x_i)} \ell(f_w(x'_i), y_i)$$

- Inner problem: data space. Outer problem: parameter space
- Why solving min-problem simple? Good landscape.
- Why solving min-max problem hard? Bad landscape.
- No matter what solution (smoothing, ensemble, etc.), the **min-max landscape must be nice!** (though indirectly imposing it)
- **Random smoothing lesson:** designing **data-space landscape** is efficient!

Fundamental Math

- Why is optimization over w easy, but over x hard?

$$\min_w \sum_i \max_{x'_i \in B(x_i)} \ell(f_w(x'_i), y_i)$$

- $f(x, w, v) = (1 - w h(v, x))^2$.
- Claim: for (w, v) space, there is no bad basin. [Li et al.'18]
- Claim: for x space, there is bad basin!
- Thus, random smoothing over x is very useful!

Future Directions

- Explaining why adversarial examples exist (still not clear).
- Smoothing: better ways of smoothing (training x , w jointly)
- Exploring parameter space landscape (current exploration only scratches the surface)
- Designing better min-max landscape?
- Combining geometry with other perspectives, e.g., information — geometry is never an isolated subject

Thank you for listening!